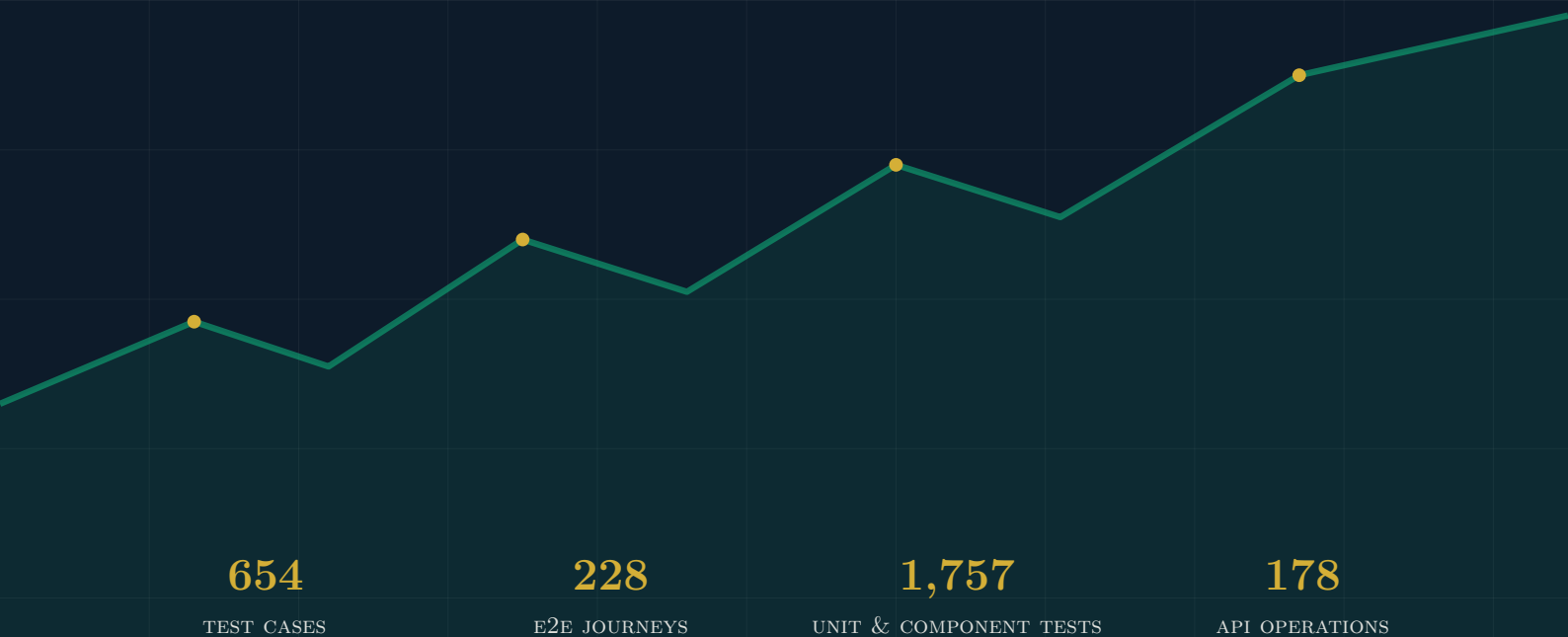


NafaIQ

Software Test Plan

Test strategy, design and execution for the NafaIQ investing and personal-finance platform — web, mobile and API.



Contents

1	Introduction	2
1.1	Purpose	2
1.2	Product overview	2
2	Test items and scope	2
2.1	Items under test	2
2.2	Feature modules	2
2.3	Out of scope	3
3	Test strategy overview	3
4	Unit and component testing	4
5	Integration and contract testing	4
6	Automated end-to-end testing (Playwright)	4
6.1	Design rules	5
6.2	Live failure replay	5
7	API testing (Postman)	5
8	Manual, exploratory and adversarial testing	6
9	Test design and documentation	6
10	Test environment and data	6
11	Continuous integration and nightly runs	6
12	Defect management	7
13	Entry and exit criteria	7
14	Risks and mitigations	8
15	Deliverables and results snapshot	8

Product	NafaIQ — PSX investing and personal-finance platform (web, mobile, API)
Prepared by	Team-H (Quality Assurance)
Test basis	Live application (dev branch), OpenAPI schema (178 operations), product feature set
Defect tracker	Jira, project KAN
Companion artefacts	NafaIQ_Test_Cases_.xlsx (test cases and results), postman/ (API collections), frontend/packages/e2e/ (automated journeys), .github/workflows/ (CI gates)

1 Introduction

1.1 Purpose

This document is the master test plan for the NafaIQ platform. It defines what is tested, at which level, with which tools and data, in which environments, and how results and defects are recorded. It is the first of the five QA-week deliverables and the frame of reference for the other four:

1. **Test plan** — this document.
2. **Test cases covering all features** — NafaIQ_Test_Cases_.xlsx: 654 cases across 25 feature modules plus the automated-journey results sheet.
3. **Automated end-to-end test cases** — the Playwright suite (@nafaiq/e2e): 228 tests in 20 files across three browser projects, wired into CI and a nightly run, with a live failure-replay harness (Section 6).
4. **Bugs reported in Jira** — defects filed to project KAN under a fixed reporting convention (Section 12).
5. **API tasks** — two Postman collections: a complete 178-operation API catalog and a 68-request scripted end-to-end QA suite (Section 7).

1.2 Product overview

NafaIQ is a Pakistan-focused investing and personal-finance product covering live PSX market data and analysis (signals V4), portfolio tracking, personal finance (transactions, budgets, bills, goals, zakat), price/budget/goal alerts, notifications, an AI assistant and AI reports, a learning hub with quizzes and an AI tutor, and an admin subsystem. It ships as a React web app, an Expo (React Native) mobile app, and a FastAPI backend on a Supabase (PostgreSQL) database. The interface is bilingual (English and right-to-left Urdu).

2 Test items and scope

2.1 Items under test

- **Web app** (frontend/packages/web) — all user-facing routes, public and authenticated.
- **Backend API** (backend/) — all 178 operations enumerated from the live OpenAPI schema, including the 35-endpoint admin subsystem.
- **Mobile app** (frontend/packages/mobile) — screens, hooks and API clients at unit/component level.
- **Cross-cutting qualities** — security and authorization, data integrity, localization/RTL, accessibility, PWA behaviour and resilience, performance and session handling, AI guardrails.

2.2 Feature modules

Test cases are organised into 25 modules, each on its own workbook sheet with a stable ID prefix:

Module	ID prefix	Examples of coverage
Authentication	TC-AUTH	sign-up/sign-in, validation, session, demo account
Dashboard	TC-DASH	greeting, KSE-100 strip, quick-add flows, AI recommendation
Portfolio	TC-PORT	holdings CRUD, cost averaging, sell paths, oversell guard
Finance	TC-FIN	transactions, budgets, bills, goals, summaries
Alerts / Notifications	TC-ALERT, TC-NOTIF	all condition types, channels, history
Watchlist	TC-WATCH	add/remove, duplicates, persistence
Market / Stock Detail	TC-MKT, TC-STK	index, movers, symbol pages
Market Data & Macro	TC-MKTD	screeners, sectors, macro data
Market Analysis V4	TC-MA	signals endpoints, caching, degradation
AI Assistant / Reports	TC-AI, TC-REP	chat, streaming, quotas, report generation
Learn Hub	TC-LEARN	lessons, quizzes, AI tutor
User Profile	TC-PROF	settings, preferences, currency, theme
Admin Dashboard	TC-ADM	RBAC, users, roles, audit, flags
Security & Authorization	TC-SEC	token handling, IDOR, credential scope
Data Integrity	TC-DATA	zakat correctness, arithmetic reconciliation, bulk delete
Cross-Module Flows	TC-FLOW	journeys that span portfolio, finance and alerts
Localization & RTL	TC-L10N	Urdu interface, RTL layout, translations
Accessibility	TC-A11Y	WCAG checks, keyboard, labels, skip links
PWA & Resilience	TC-PWA	offline behaviour, install, recovery
Performance & Session	TC-PERF, TC-SESS	latency, caching, session expiry
AI Guardrails	TC-AIG	write confirmation, cross-user leakage, prompt abuse
Monetary Desk	TC-MON	monetary/macro desk views

2.3 Out of scope

- Load and stress testing beyond the light concurrency probes recorded in Performance & Session.
- Native mobile end-to-end automation on physical devices (mobile is covered at unit/component level plus a Pixel 7 mobile-viewport browser project in the E2E suite).
- Third-party systems themselves (Supabase, PSX data provider, AI model providers, e-mail delivery) — only NafaiQ’s integration with them.
- Subjective quality of AI-generated prose (factual/guardrail behaviour *is* in scope under AI Guardrails).

3 Test strategy overview

Testing is layered so that each class of defect is caught by the cheapest layer able to catch it, and every layer is automated where determinism allows. Sections 4–8 define each layer in detail.

Layer	Tooling	Volume	Primary target
Unit & component	pytest, Vitest, Jest	1,757 tests	logic, components, hooks, stores
Integration & contract	pytest overrides, MSW	within the above	wiring, error taxonomy, schema drift
End-to-end (automated)	Playwright	228 executions	real user journeys in a real browser
API (manual + scripted)	Postman	178 + 68 requests	full API surface, hard journeys
Manual & adversarial	live probes	319 executed cases	security, data integrity, judgement calls

Every defect fix is re-verified at the level that found it, and the full backend suite plus the Playwright suite are re-run after each fix wave. CI (Section 11) makes this automatic for every subsequent change.

4 Unit and component testing

- **Backend** — pytest: **1457 passed, 4 skipped** at plan time, across 144 test files. API routes are exercised offline through dependency overrides and fake repositories; live-database tests auto-skip when credentials are absent so CI never depends on Supabase.
- **Web** — Vitest with Testing Library and MSW: **179 tests in 18 files** covering components, hooks, Redux slices and API contracts.
- **Mobile** — Jest (jest-expo) with React Native Testing Library: **121 tests in 19 suites**; every screen test also asserts accessibility roles/labels per the mobile conventions.

Coverage is risk-ranked rather than exhaustive: money paths (portfolio arithmetic, finance summaries, zakat), authentication and authorization, alert evaluation, and the AI quota/refresh invariants take precedence over cosmetic rendering.

5 Integration and contract testing

Backend route→service→repository wiring is tested with the same offline override pattern used at unit level, extended to whole-request flows:

- **Cross-user isolation** — every owned resource is probed with another user’s identity; the expected result is a scoped 404, never a 403 that confirms existence.
- **Error taxonomy** — 401/403/404/422/429 contracts are asserted per route family, including the exact 401 messages the client refresh flow distinguishes.
- **Pagination and boundary edges** — empty datasets, last-page behaviour, inclusive boundaries (e.g. the zakat nisab threshold).
- **Frontend contracts** — web and mobile API mocks are generated from the real backend response schemas, so a backend field rename fails a frontend test rather than production. This guard exists because the finance-summary contract drifted once before QA week.
- **Policy middleware** — CORS is pinned by test: allowed production and loopback origins, and anchored-regex rejection of `localhost.evil.com`-style suffix attacks.

6 Automated end-to-end testing (Playwright)

The `@nafaiq/e2e` package runs **228 tests in 20 files** against a real browser and the real application stack (Vite dev server plus a local uvicorn backend), across three projects:

Project	Browser	User
public-chromium	Desktop Chrome	Anonymous
authed-chromium	Desktop Chrome	Demo user
authed-mobile	Pixel 7 viewport	Demo user

6.1 Design rules

All of the following are load-bearing for determinism:

- **Journeys only** — every spec is a user journey (open, act, observe on the destination page), asserted with web-first matchers on structure rather than polled market values.
- **One real login** — a setup project signs in once with the demo account and saves storage state; only the Supabase session keys are persisted so no spec leaks app state into the next.
- **Deterministic AI** — AI, assistant, tutor and report endpoints are stubbed with canned responses in the exact streaming (SSE) framing the client parses.
- **Console guard** — any unexpected browser console error fails the test; known product bugs are allow-listed with a remove-when-fixed note tied to their Jira key.
- **Auth-settle guard** — no interaction starts until the auth-resolution overlay has cleared, eliminating the swallowed-tap class of mobile flake (root cause of an early Pixel 7 defect).
- **Demo-account safety** — demo-user writes live entirely in local state; browser journeys therefore never mutate shared backend data.

6.2 Live failure replay

A hard failure in the automated suite can be reproduced *in front of the tester* without manual setup. `pnpm test:e2e:live` runs the full suite and, on the first hard failure, launches a headed Chromium, signs in through the visible NafaIQ form using the stored demo credentials, and re-executes the failing journey:

- If the failure **reproduces**, Playwright pauses indefinitely with the browser and Inspector open at the failure state for live debugging.
- If it **passes** (a flake), the end state stays visible for five seconds and the browser closes.

`pnpm test:e2e:replay` replays any chosen failure from the latest JSON report by index or by fingerprint. Replay mode disables traces and uses only the console reporter so the demo password never lands in an artefact, and the original suite report is not overwritten.

7 API testing (Postman)

Two collections, kept in `postman/` with Local and Dev environments:

- **NafaIQ API — Complete Catalog:** all **178** FastAPI operations, generated from the live OpenAPI schema and organised by the 27 API areas (finance, portfolio, signals-v4, admin, ...). Every request carries an assertion script; write requests are prefixed *REVIEW BEFORE SEND*. This is the discovery and manual-inspection surface.
- **NafaIQ API — End-to-End QA: 68** scripted requests in nine ordered journey folders (setup/authentication, public market, portfolio, watchlist, personal finance, goals/budgets/bills, alerts and notifications, negative and authorization tests, optional AI). Requests chain dynamically captured IDs, assert on status and body, and clean up created records where the API supports it, so the collection is re-runnable and presentation-safe.

The catalog is also the coverage yardstick: it exposed that only ~40 of 155 user-facing endpoints had cases before the expansion pass, which drove the creation of the 13 newest workbook modules.

8 Manual, exploratory and adversarial testing

Cases that automation cannot decide are executed manually against a live local backend with real JWTs, with observed evidence recorded in the workbook's *Actual Result* column. The adversarial pass focused on: security and authorization (RBAC on all admin endpoints, IDOR across two real accounts, credential-scope confusion, token error hygiene), data integrity (zakat nisab authority, clamping and boundaries, portfolio arithmetic reconciliation, cost-averaging), input validation (impossible dates, negative quantities, oversell), and AI guardrails (write-confirmation, cross-user data requests, Roman-Urdu queries). Destructive probes were pointed at disposable records or deliberately blocked (e.g. the untested bulk-delete endpoint, TC-DATA-20, was not fired at the shared demo account).

9 Test design and documentation

All manual and API test cases live in `NafaIQ_Test_Cases_.xlsx`:

- **One sheet per module**, uniform columns: ID, module, scenario, preconditions, steps, test data, expected result, actual result, status, priority.
- **Statuses**: Pass, Fail, Partial, Blocked. A *Blocked* case is never left bare — its Actual Result states exactly what it needs (device, admin account, market window, failure injection, seeded data, human judgement). Nothing is marked Pass without recorded evidence.
- **Priorities**: P1 (core money paths, security, data integrity) through P4 (cosmetic). Current spread: 221 P1, 211 P2, 66 P3, 7 P4.
- **E2E Journey Results sheet**: one row per automated journey execution (149 rows = journeys × browser projects), carrying the automation source file, evidence path, run commit and timestamp.
- **Execution Summary sheet**: run metadata, defect focus, and the audit trail of each execution pass.

Traceability is bidirectional: automated cases name their spec file in the workbook row, and defects name their test case in Jira.

10 Test environment and data

- **Local stack** — Vite dev server (web) + unicorn (backend) + Supabase project; this is what Playwright boots and what manual API execution targeted (127.0.0.1:8000).
- **Deployed** — backend on Railway, web on Vercel; the Postman Dev environment and an `E2E_API_URL` escape hatch point automation at it.
- **Test identities** — the shared **demo account** (read-mostly by design; its writes stay client-side), a **second real account** for cross-user/IDOR checks, a purpose-made **QA super-admin** (role granted in the database, proving roles are DB-resolved rather than token-derived), and disposable targets for destructive operations. The demo account is deliberately *not* an admin, because several cases assert it is refused.
- **Data hygiene** — every server-side write in manual execution went to a throwaway record and was cleaned up; the demo baseline (net worth 164,467 across 3 holdings) was re-verified identical after the pass.

11 Continuous integration and nightly runs

- **ci.yml** — on every pull request and push: lint/typecheck, web unit, mobile unit, back-end tests, and the two Chromium E2E projects, converging on a single required check (**ci-required**) that branch protection and deploy gates key on.

- **nightly-e2e.yml** — weekdays 08:00 PKT (before PSX opens): the full three-project matrix including the Pixel 7 mobile viewport, which is deliberately off the merge path. The nightly **always checks out the dev branch** (the integration branch), so a green nightly means anything promoted **dev**→**main** is already QA-tested; a manual dispatch can override the ref.
- **Branch flow** — feature branches → **dev** (integration) → **main** (production), with **ci-required** green as the promotion condition.

12 Defect management

Defects are filed in Jira project KAN as they are found, under a fixed convention:

- **Title:** plain words stating what is broken and where.
- **Description:** exactly three parts — *Issue* (1–2 sentences), *Steps* (short reproduction bullets), *Expected* (1–2 sentences). Machine context (fingerprints, evidence paths, repro commands) goes in labels or a comment, never the description.
- **Severity** is judged by user impact and data/monetary risk; religiously sensitive computation (zakat) is automatically high.
- **Automated triage:** a runner parses the Playwright JSON report and files hard failures to KAN with a stable fingerprint label per failure, so re-occurrences comment on the existing ticket instead of duplicating it; infrastructure failures collapse to a single ticket, and a flood of new fingerprints halts for human confirmation.

Representative defects from this cycle (all found by planned cases, all but one fixed and re-verified within the week):

Jira	Severity	Defect	State
KAN-52	Highest	Zakat trusted a client-supplied nisab; nisab=0 made PKR 1,000 of assets zakat-liable	Fixed
KAN-51	High	Stock screener returned 500 on every request (service module missing)	Fixed
KAN-56	High	Admin role history crashed for roles granted by a real admin	Fixed
KAN-53	Medium	Add-holding accepted future and impossible purchase dates	Fixed
KAN-57	Medium	Duplicate React keys on every lesson page (breadcrumb)	Fixed
KAN-58	Medium	Missing skip-to-content link; unlabelled form fields (WCAG)	Fixed
KAN-55	Medium	Market-analysis endpoint fully public — gating is a product decision	Open
KAN-54	Low	Auth 401 leaked Python codec internals to unauthenticated callers	Fixed

Verified-correct behaviour is recorded with the same rigour as defects (RBAC, IDOR scoping, oversell protection, cost averaging, injection safety, zakat clamping), so the presentation can state what is *known good*, not only what broke.

13 Entry and exit criteria

Entry (per execution pass): the build under test runs locally end to end; test data baseline recorded; required identities available.

Exit (met at plan time):

- Every workbook case carries a real result or a specific blocking reason — no case left silently unexecuted.
- All automated suites green: backend 1457 passed, web 179, mobile 121, Playwright 228 executions across three projects (latest nightly: all three projects passing).
- No open Highest/High defect without a fix or an explicit product decision pending (sole remaining Fail: KAN-55, awaiting a gating decision).
- CI and nightly gates active on the integration branch.

Suspension: execution against shared data suspends immediately if a probe mutates the shared demo baseline; the baseline is restored and verified before resuming (exercised once during the pass, restored to the exact prior state).

14 Risks and mitigations

Risk	Mitigation
Live PSX market data makes assertions time-of-day dependent	Journeys assert structure, not values; market-hours recorded fixtures planned for the still-blocked chart cases
Shared demo account could be polluted by test writes	Demo writes are client-side by design; server-side probes use throwaway records and a verified restore
AI output is non-deterministic	AI endpoints stubbed in E2E; guardrail behaviour (not prose quality) tested live
Flaky E2E failures erode trust in the gate	Retries in CI classify transients as flaky; live replay distinguishes real failures from flakes in one command; console guard keeps hidden errors visible
Concurrent merges can turn two green branches red together	The push-triggered CI run on the integration branch tip is the verdict of record; nightly re-checks <code>dev</code> daily
Blocked cases silently rot	Every Blocked row names its unblocking requirement, bucketed (device, admin, market window, failure injection, ...) for planned follow-up

15 Deliverables and results snapshot

Deliverable	Artefact	Status at plan time
Test plan	this document	Done
Test cases, all features	NafaiQ_Test_Cases_.xlsx: 654 cases, 25 modules + journey results	412 Pass / 8 Partial / 1 Fail / 233 Blocked-with-reason; 319 executed live this week
Automated E2E	@nafaiq/e2e: 228 tests, 20 files, 3 browser projects + live replay	Green locally, in CI and in the nightly (3/3 projects)
Bugs in Jira	project KAN	Defects filed throughout; this pass: 8 filed, 7 fixed and re-verified, 1 awaiting product decision
API tasks (Postman)	postman/: 178-operation catalog + 68-request scripted QA journeys, Local/Dev environments	Done; QA collection re-runnable with cleanup

Supporting (beyond the asked deliverables): 1757 unit/component/integration tests across the three codebases, CI gating on every pull request, a nightly cross-viewport run pinned to the integration branch, and an automated Playwright-failure-to-Jira triage path.